

D'un problème théorique à un jeu de logique : génération et résolution du Slither Link

Nohlan Saucet

7 juin 2026

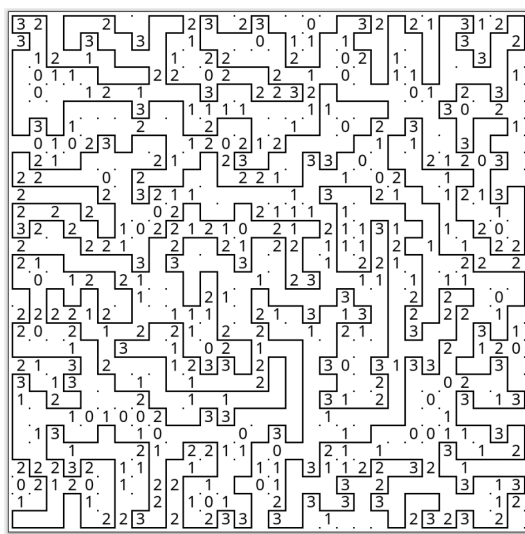


FIGURE 1 – Grille de Slither Link

Résumé

Le Slither Link, puzzle édité par Nikoli, allie des règles simples à une nature NP-complet. Dans ce travail, nous explorons les deux facettes du problème : la génération automatique de grilles valides et intéressantes, ainsi que leur résolution par des algorithmes de retour sur trace et des solveurs SMT. Nous présentons une modélisation par 2-coloration basée sur le théorème de Jordan, un critère de modifiabilité des cases et diverses méthodes de génération (grignotage, crevasses, parcours aléatoire, mutation). Côté résolution, un algorithme de backtracking original optimisé résout des grilles 200×200 en deux minutes lorsque toutes les cases contiennent un indice. Nous discutons la preuve de NP-complétude due à Yato, établissons une correspondance rigoureuse entre colorations et cycles simples dans le graphe dual, et généralisons les résultats aux graphes planaires (CS-colorations). Des perspectives vers des puzzles à plusieurs couleurs et en trois dimensions sont enfin esquissées.

Table des matières

1	Introduction	3
2	Présentation du Slither Link	4
2.1	Règles du puzzle	4
2.2	Modélisation par coloration et théorème de Jordan	4
2.3	Différentes implémentations en Python	5
3	Génération de puzzles	6
3.1	Approche naïve : marche aléatoire auto-évitante	6
3.2	Critère de modifiabilité d'une case	6
3.3	Méthodes de génération par grignotage	7
3.4	Autres méthodes de génération	9
4	Résolution algorithmique	10
4.1	Naïveté et explosion combinatoire	10
4.2	Méthode Gloutonne	10
4.3	Backtracking original	12
4.4	SMT solvers et comparaisons	15
5	Généralisations et perspectives	16
5.1	CS-colorations et génération de cycles simples	16
5.2	Application : Slither Link sur graphe planaire	18
6	Preuves et résultats théoriques	19
6.1	Démonstration du critère de modifiabilité généralisé	20
6.2	Correspondance cycle / CS-coloration	21
6.3	NP-complétude du Slither Link	22
7	Conclusion	25

1 Introduction

Le Slither Link (Slitherlink), publié par Nikoli, est un puzzle de logique dont les règles sont d’une sobriété trompeuse : sur une grille de points, tracer une boucle fermée, sans croisement ni branchement, de sorte que chaque case numérotée possède exactement autant de côtés bordés par le tracé que l’indique son chiffre. Cette apparente simplicité dissimule une complexité fondamentale — le problème de résolution est NP-complet [1] — et c’est précisément cette tension qui motive le présent travail. Fruit d’une année de recherche, cet article propose une étude approfondie du Slither Link selon deux axes : la génération de puzzles valides et esthétiques, et leur résolution algorithmique. Une seule notion reste en marge : celle de l’unicité de la solution, dont les deux contributions majeures sont l’ASP-complétude de Yato [1] et les Slitherlink Signatures de Beluhov [7].

Après un rappel des règles (Section 2.1), nous présentons plusieurs approches de génération (Section 3) : marche aléatoire auto-évitante, méthode du coloriage intérieur/extérieur, critère de modifiabilité et améliorations (grignotage, crevasses, mutation). La Section 4 compare un algorithme de résolution glouton, un algorithme de retour sur trace efficace et l’emploi de solveurs SMT. Enfin, la Section 5 ouvre vers la génération de cycle auto-évitant dans des graphes planaires quelconques et son application pour généraliser la génération de puzzle Slither Link. La Section 6 contient la démonstration du critère de modifiabilité généralisé aux graphes planaires, la correspondance entre cycles et CS-colorations, ainsi qu’une présentation détaillée de la NP-complétude.

L’ensemble des programmes utilisés sont disponible sur cette page [Github](#)

2 Présentation du Slither Link

2.1 Règles du puzzle

Éléments clés :

- Grille de points ; on relie des segments horizontaux ou verticaux.
- Objectif : former une unique boucle fermée, sans croisement ni branchement.
- Contraintes : les nombres de 0 à 3 dans certaines cases indiquent le nombre de segments adjacents à cette case.

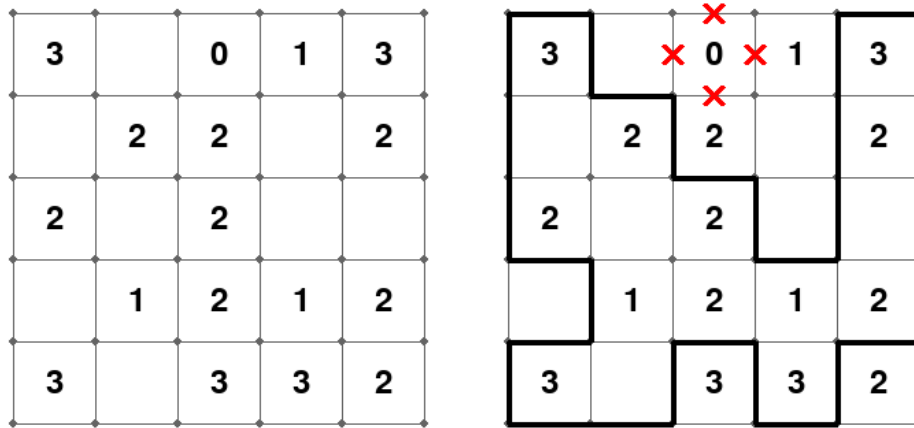


FIGURE 2 – Exemple de grille Slither Link (gauche : grille non résolue ; droite : solution avec la boucle fermée).

Le Slither Link se joue sur une grille rectangulaire de *points* de taille $(n+1) \times (m+1)$. Les segments (arêtes) horizontaux ou verticaux entre points adjacents peuvent être tracés ou non. La contrainte vient des *cases* de taille unitaire : certaines cases contiennent un entier $c \in \{0, 1, 2, 3\}$; la boucle doit entourer exactement c des quatre côtés de cette case. La solution doit être un *cycle simple* (aucun sommet de degré $\neq 2$ dans le sous-graphe tracé) et connexe. La figure 2 montre une grille 5×5 de cases (donc 6×6 points) avec des indices et la boucle solution.

2.2 Modélisation par coloration et théorème de Jordan

Une avancée décisive consiste à utiliser le théorème de Jordan : tout cycle simple dans le plan sépare celui-ci en une région intérieure et une région extérieure. Par conséquent, on peut colorier chaque *case* en INTÉRIEUR (I) ou EXTÉRIEUR (E) selon qu'elle se trouve à l'intérieur ou à l'extérieur de la boucle. Le problème se réduit alors à :

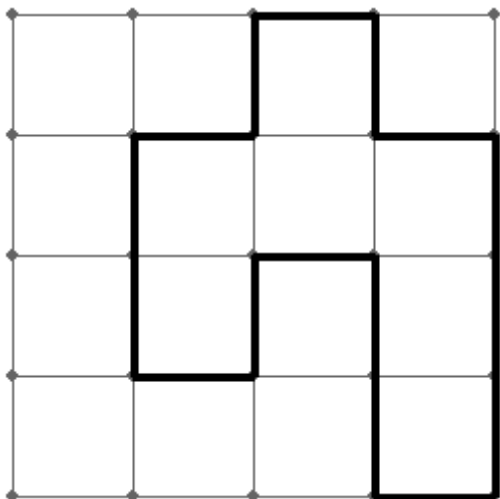
- Une matrice $C_{i,j} \in \{I, E\}$ pour chaque case.
- Un nombre (indice) $d_{i,j} \in \{-1, 0, 1, 2, 3\}$ où -1 signifie l'absence d'indice.

La contrainte sur un indice c se traduit par : parmi les quatre cases voisines (nord, est, sud, ouest) de la case (i, j) , exactement c doivent être de couleur différente de celle de (i, j) (car une arête est tracée si et seulement si elle sépare une case intérieure d'une case extérieure).

2.3 Différentes implémentations en Python

Dans la suite de l'article, on modélise d'abord un *puzzle* de taille $n \times m$ par une matrice de taille $n \times m$ où la valeur en (i, j) est l'indice de la case s'il existe, et -1 sinon. Ensuite, on s'intéresse aux deux modélisations d'une *solution* qui découlent naturellement :

La première définit une solution par deux matrices : l'une représente les arêtes horizontales, l'autre les verticales. Pour chaque arête, les matrices indiquent si elle appartient ou non au cycle qui forme la solution.

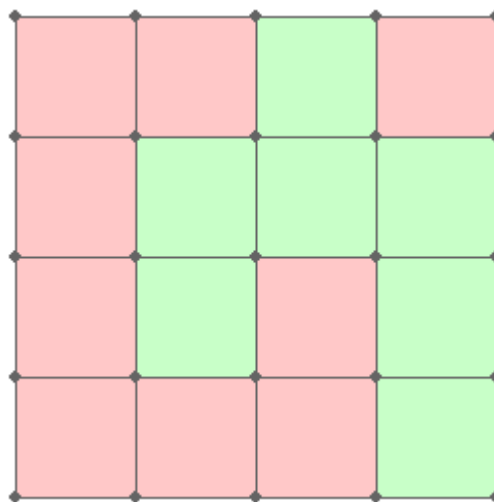


```

1 F, T = False, True
2
3 Edges_horizontal = [[F,F,T,F],
4                     [F,T,F,T],
5                     [F,F,T,F],
6                     [F,T,F,F],
7                     [F,F,F,T]]
8
9 Edges_vertical = [[F,F,T,T,F],
10                  [F,T,F,F,T],
11                  [F,T,T,T,T],
12                  [F,F,F,T,T]]

```

La seconde définit une solution par une unique matrice qui indique si une case est à l'intérieur (I) ou à l'extérieur (E) de la boucle.



```

1 E, I = False, True
2
3 Map = [[E,E,I,E],
4        [E,I,I,I],
5        [E,I,E,I],
6        [E,E,E,I]]

```

Il est aisé de passer d'une représentation à l'autre. Cependant, la seconde modélisation est bien plus compacte algorithmiquement : il suffit de manipuler deux matrices $n \times m$. De plus, elle fournit un critère de modifiabilité locale pour savoir si la couleur d'une case peut être changée sans briser la propriété de cycle simple. Nous la détaillons dans la sous-section 3.2.

3 Génération de puzzles

Pour générer un *puzzle* Slither Link, il est vain de placer des chiffres aléatoirement sur une grille. Il faut construire le puzzle à partir d'une solution. Générer une grille de Slither Link demande de construire une solution, c'est-à-dire un cycle simple aléatoire sur une grille. Différentes approches sont proposées et étudiées dans cette section.

3.1 Approche naïve : marche aléatoire auto-évitante

Une idée naturelle est d'utiliser la première modélisation de la solution et de simuler une marche aléatoire sur $\mathbb{Z}^2$ avec croisement jusqu'à revenir à l'origine (ce qui est certain par une étude probabiliste). Pour une grille $n \times m$ de cases, on travaille sur les points $(n+1) \times (m+1)$, puis on modifie le chemin pour éviter les croisements. Mais cette méthode souffre de plusieurs écueils :

- La probabilité de former un cycle simple de grande taille est très faible.
- Il faut ensuite éviter de créer des composantes connexes multiples (boucle unique).
- La représentation par trois matrices (points, arêtes verticales, arêtes horizontales) est lourde à manipuler en Python.

Dans la suite, on s'intéresse au cas où l'on utilise la seconde modélisation. Avant cela, on introduit le critère qui simplifie la génération de solutions par cette modélisation.

3.2 Critère de modifiabilité d'une case

Définition 1 (Case modifiable). *Soit une case X de couleur c_X (I ou E). Notons $\mathcal{C}$ l'ensemble des huit cases voisines (les quatre cases orthogonales et les quatre cases diagonales). Si une de ces cases n'existe pas, elle est remplacée par une case fictive de couleur E , de sorte que $|\mathcal{C}| = 8$. On dit que X est modifiable si :*

1. *X est adjacente (horizontalement ou verticalement) à au moins une case de couleur I et une case de couleur E .*
2. *Dans le sous-graphe formé par les cases de $\mathcal{C}$, les cases de couleur I forment une seule composante connexe (par adjacence orthogonale), et de même pour les cases de couleur E .*

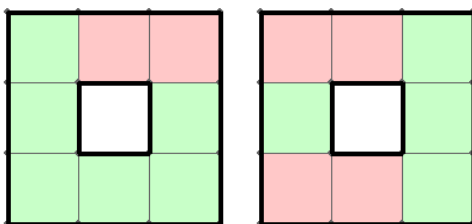


FIGURE 3 – Illustration du critère de modifiabilité : à gauche, la case centrale est modifiable ; à droite, le critère 2 n'est pas vérifié, donc la case centrale n'est pas modifiable.

L'intérêt pratique est que l'on peut modifier la couleur d'une case modifiable sans casser la propriété d'unicité du cycle dans la solution. Cela permet de générer aléatoirement des puzzles en partant d'un cycle initial simple (par exemple un rectangle) et en appliquant des retournements de cases modifiables. On obtient ainsi des boucles sinueuses et variées.

3.3 Méthodes de génération par grignotage

Grignotage carré À partir du bord de la grille, on “mange” aléatoirement des cases modifiables tout en préservant l’existence d’un cycle.

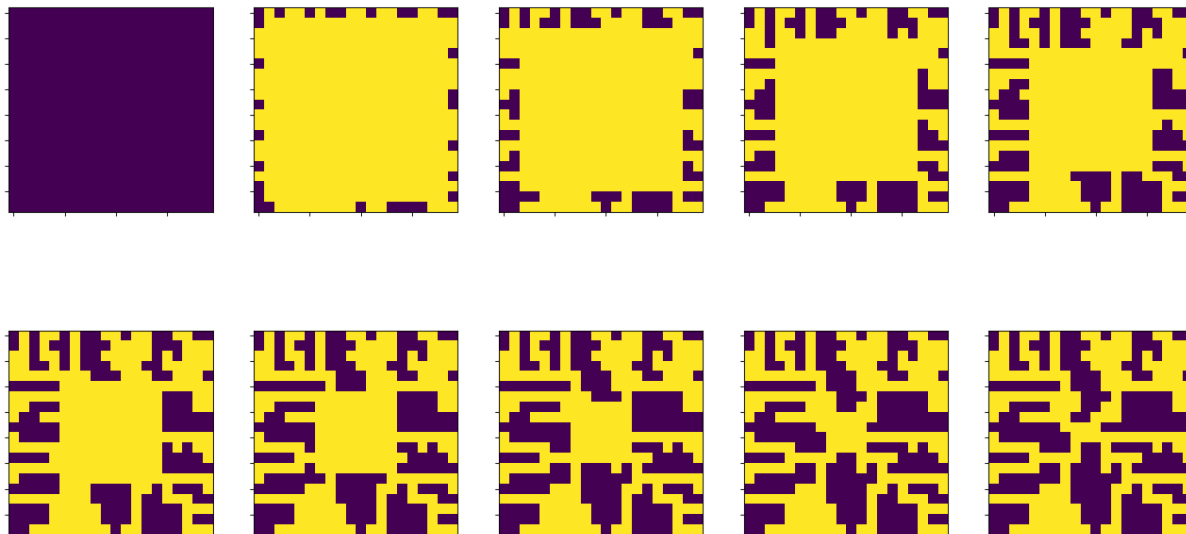


FIGURE 4 – Algorithme de grignotage pour une solution de taille 20×20

En modifiant la probabilité de modifier une case modifiable à chaque étape, la forme de la solution s’en trouve influencée :

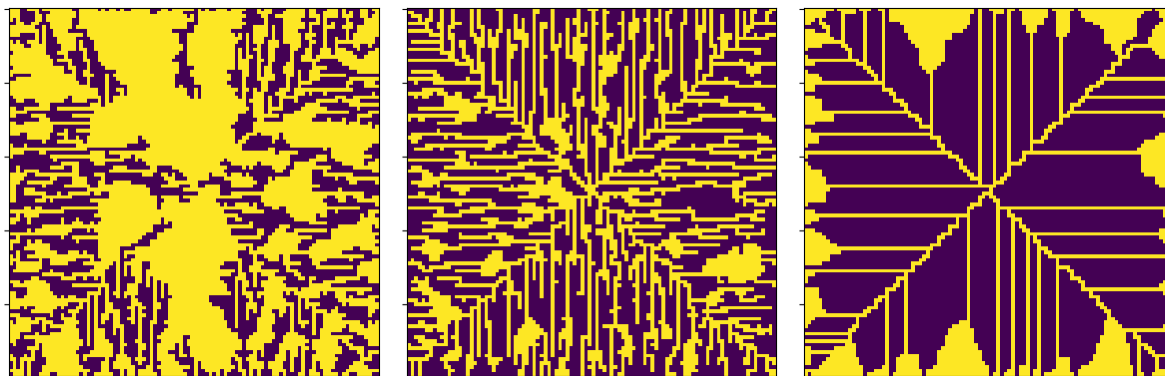


FIGURE 5 – Algorithme de grignotage 100×100 : 60%, 80%, 100% des cases du contour à chaque étape

Cette méthode efficace ($\mathcal{O}(n \times m)$) présente quelques défauts :

- La génération peut laisser de grandes taches intérieures ou extérieures, ce qui équivaut à un grand nombre de 0.
- Un motif de croix sur les diagonales apparaît dans les solutions générées.
- La case centrale est toujours coloriée en intérieur.

Ces motifs récurrents ne sont pas souhaitables pour former des puzzles originaux et difficiles. Nous proposons alors une amélioration de cet algorithme.

Grignotage carré + crevasse On insère des crevasses avant d'appliquer l'algorithme de grignotage carré.

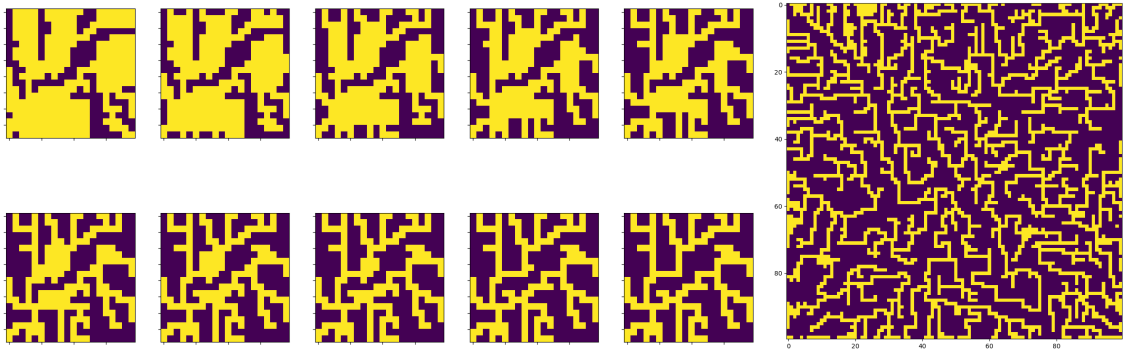


FIGURE 6 – Algorithme de grignotage 20×20 carré avec crevasse, étape par étape, ainsi qu'une solution de taille 100×100 carré avec crevasse

Cela évite les problèmes énoncés avec la méthode précédente, mais laisse encore beaucoup d'espace extérieur. Dans ce cas, le puzzle a une forme satisfaisante et garde une complexité en $\mathcal{O}(n \times m)$. Ainsi, en jouant sur le nombre de crevasses et la probabilité de modification, il est tout à fait envisageable d'obtenir des puzzles intéressants et variés. On propose finalement une génération plus gourmande mais avec des résultats très satisfaisants sans réglage fin des paramètres.

Grignotage récursif Cet algorithme commence de la même manière que le grignotage simple, sauf que lorsqu'on modifie une case, une fonction récursive est appelée. Cette fonction regarde aléatoirement un voisin de la case et, si ce voisin est modifiable, alors la case est modifiée et l'algorithme est rappelé sur la case suivante, jusqu'à ce que, soit la case choisie ne soit pas modifiable, soit la fonction dépasse un nombre N de récursions fixé à l'avance. Dans la plupart des cas, cet algorithme semble donner des résultats plus satisfaisants.

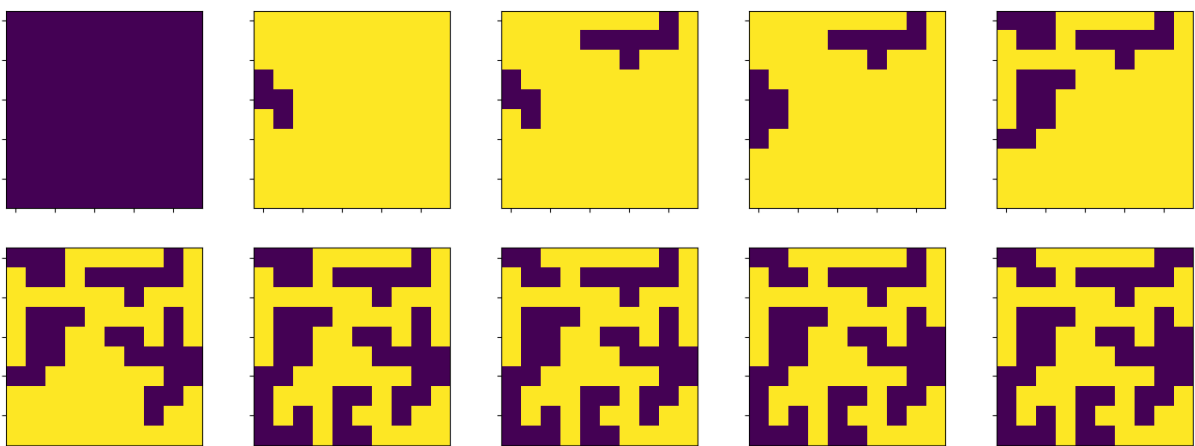


FIGURE 7 – Algorithme de grignotage récursif 10×10

3.4 Autres méthodes de génération

Génération par labyrinthe Il est tout à fait possible d’imaginer générer un puzzle à partir d’un labyrinthe parfait (il existe un unique chemin d’un point A à un point B qui ne passe pas deux fois par le même sommet) à l’aide de l’algorithme de Kruskal ou de l’algorithme de Prim. Même si le résultat est plus régulier, cela reste intéressant.

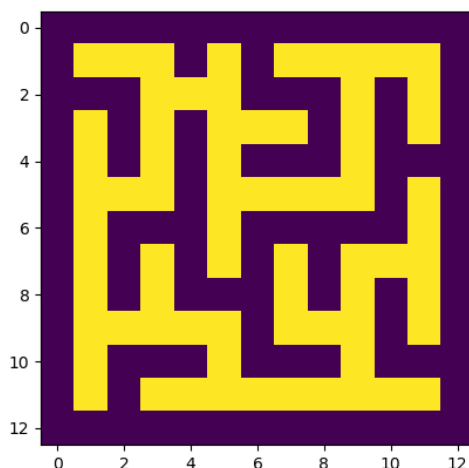


FIGURE 8 – Exemple de génération de solution 13×13 par labyrinthe

Génération par mutation On peut, à chaque itération, faire la liste de toutes les cases modifiables, en modifier une aléatoirement, et recommencer N fois la boucle. Cette méthode peut fournir des résultats satisfaisants mais est gourmande en ressources (actuellement $\mathcal{O}(n \times m \times N)$). Une grande amélioration consisterait à modifier localement la propriété de modifiabilité après chaque changement au lieu de reparcourir toute la matrice.

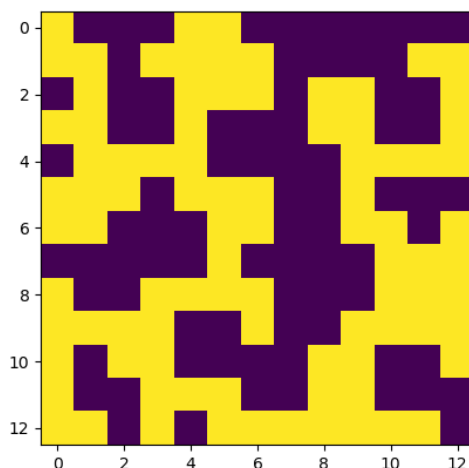


FIGURE 9 – Exemple de génération de solution 13×13 avec 100 mutations

Génération par brute force On énumère exhaustivement (sur de petites grilles) toutes les solutions possibles de taille $n \times m$, puis on définit des critères esthétiques basés sur le nombre de cases modifiables, le nombre de bouts (case qui n'a qu'un seul voisin de sa couleur), un indice mesurant à quel point les couleurs I et E sont mélangées dans la solution, ou encore le nombre de zigzags. Une fois cette base de données créée, il est possible d'obtenir des puzzles en temps constant.

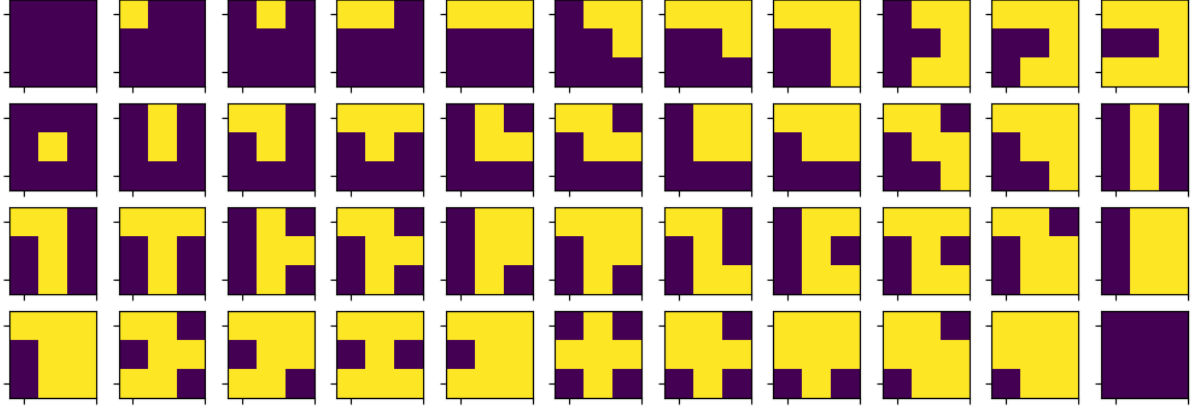


FIGURE 10 – L'ensemble des 44 types de puzzles Slither Link (à rotation et transposition près) de taille 3×3

4 Résolution algorithmique

4.1 Naïveté et explosion combinatoire

Un algorithme récursif qui essaie toutes les combinaisons de segments sur $(n + 1) \times (m + 1)$ arêtes est inutilisable dès 5×5 (environ 2^{25} possibilités). Il faut exploiter les contraintes.

4.2 Méthode Gloutonne

On propose ici une approche gloutonne du problème qui améliore l'approche brute force mais reste peu satisfaisante pour des puzzles de taille plus grande que 8×8 . On nous donne un puzzle que l'on note *Puzzle* et on munit ce puzzle d'une coloration C . L'objectif est de faire en sorte que la coloration forme une solution du puzzle.

1	3	2
1	3	2
3	2	2

1	3	2
1	3	2
3	2	2

On regarde si la coloration forme une solution. Si ce n'est pas le cas, on modifie une case parmi les 9 cases qui minimise la fonction Score définie ici :

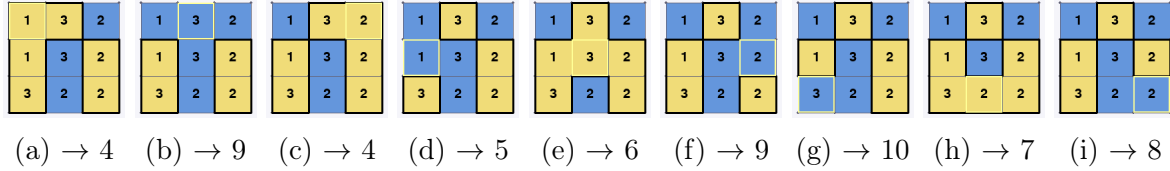
1: **function** Score(*Puzzle*, C) :

```

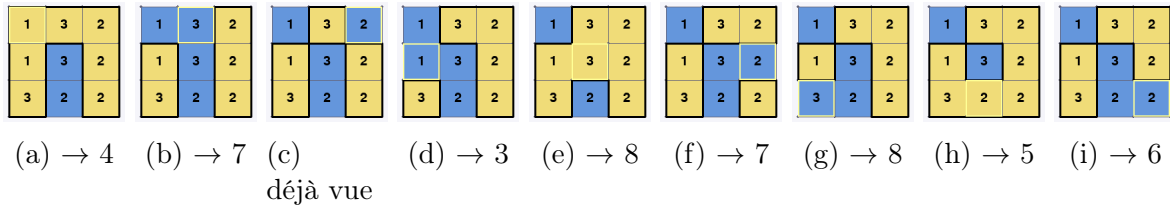
2:   $N_a \leftarrow \text{Nombre\_arrete}(C)$ 
3:   $s \leftarrow 0$ 
4:  for  $i \leftarrow 0$  to  $n - 1$  :
5:      for  $j \leftarrow 0$  to  $m - 1$  :
6:          if  $\text{Puzzle}[i][j] \neq -1$  :  $s \leftarrow s + |N_a[i][j] - \text{Puzzle}[i][j]|$ 
7:  return  $s$ 
8: end function

```

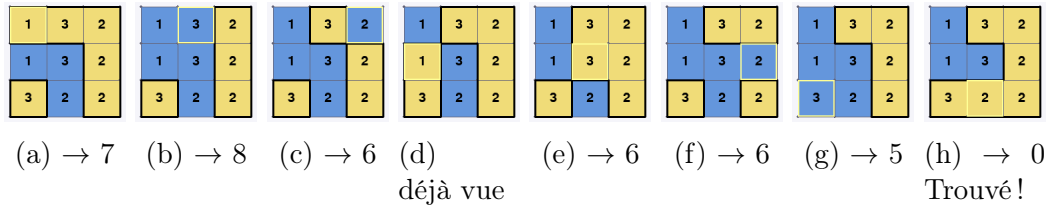
Dans l'exemple donné ci-dessus, on a :



Puisque (c) minimise le score, on réapplique la même procédure sur la coloration (c).



On réapplique la procédure sur la coloration (d).



Si l'algorithme n'arrive plus à descendre le score à partir d'une certaine étape, alors on repart d'une coloration aléatoire. Si au bout de N essais l'algorithme n'a toujours pas trouvé de solution, alors on retourne *None*.

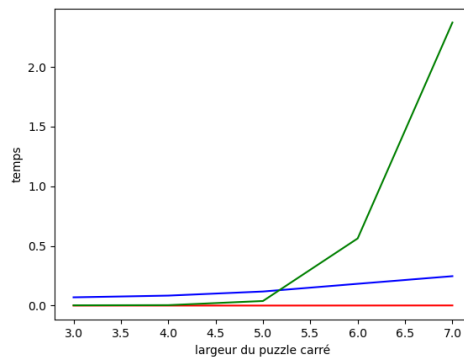


FIGURE 14 – Comparaison temps de calcul entre approche gloutonne (vert), backtracking (rouge), SMT (bleu) pour des puzzles de taille 3×3 à 7×7

4.3 Backtracking original

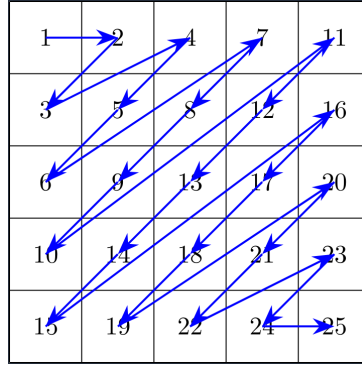


FIGURE 15 – Ordre de parcours pour le backtracking pour un puzzle 5×5

L'algorithme que nous avons implémenté parcourt les cases du puzzle en anti-diagonale comme montré sur la figure ci-contre. Le but étant de trouver rapidement des contradictions pour éviter de s'enfoncer trop profondément dans l'arbre binaire des $2^{n \times m}$ possibilités. Expliquons les conditions que nous avons construites pour déterminer au plus vite des contradictions lors de la construction. Partons d'un exemple : on souhaite résoudre un puzzle de taille 5×5 et l'algorithme observe la case $(2, 1)$.

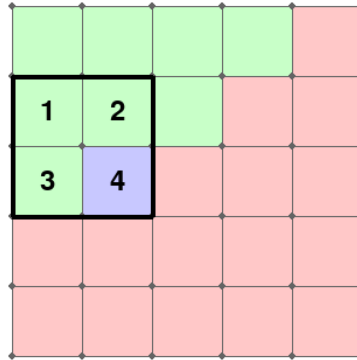


FIGURE 16 – Algorithme de backtracking en train d'observer la case $(2, 1)$ d'un puzzle 5×5

Plusieurs choses sont à noter. Premièrement, les cases en rouge ne sont pas encore déterminées et valent alors *False* (c'est-à-dire qu'elles se trouvent à l'extérieur). Deuxièmement, les cases en vert se sont déjà vu attribuer une valeur par l'algorithme (*True* si intérieur, *False* si extérieur). On remarque alors que la case notée 1 sur la figure ne peut plus être modifiée car tous ses voisins sont déjà déterminés par l'algorithme. Ainsi, son nombre de voisins de la couleur opposée doit être égal à celui indiqué par le puzzle s'il y a un indice dans cette case. La case notée 2 ne pourra plus être modifiée lors de la visite de la prochaine case. Ceci est une condition très forte sur le choix de la case : si son nombre de voisins est déjà égal à celui indiqué par le puzzle, on doit laisser la case $(2, 1)$ à *False* ; sinon on doit la modifier et attribuer la valeur *True*. Si la case notée 2 n'est toujours pas correcte, alors il y a une contradiction et on fait un retour arrière (backtrack). Dans les deux cas notés 3 et 4, les conditions sont moins fortes mais peuvent être utilisées : la case 3 ne pourra être modifiée plus qu'une fois après l'attribution de la case $(2, 1)$, et la case 4 plus que deux fois. Nous proposons ici une implémentation en pseudo-code.

Algorithm 1 backtrack_solveur_all(Na)

Require: Matrice Na des indices ($-1 = \text{inconnu}$), dimensions $n \times m$

Ensure: Liste de toutes les solutions M (booléennes) valides

```
1: function backtrack_solveur_all( $Na$ ) :
2:    $n, m \leftarrow \text{Size}(Na)$ 
3:    $M \leftarrow$  matrice  $n \times m$  de False
4:    $NaM \leftarrow$  matrice  $n \times m$  de 0
5:    $solutions \leftarrow []$ 
6:    $(i_0, j_0) \leftarrow (0, 0)$ 
7:
8:   if  $Na[i_0][j_0] \geq 2$  ou  $Na[i_0][j_0] = -1$  then
9:      $(NaM, \text{restore}) \leftarrow \text{MODIFIENAM}(M, NaM, i_0, j_0)$ 
10:    BACKTRACK( $i_0, j_0, \text{True}$ )
11:    RESTORENAM( $NaM, \text{restore}$ )
12:   end if
13:   if  $Na[i_0][j_0] \leq 2$  ou  $Na[i_0][j_0] = -1$  then
14:     BACKTRACK( $i_0, j_0, \text{False}$ )
15:   end if
16:   return  $solutions$ 
17: end function
18:
19: function Backtrack( $i, j, val$ ) :
20:    $M[i][j] \leftarrow val$ 
21:    $(NaM, \text{restore}) \leftarrow \text{MODIFIENAM}(M, NaM, i, j)$ 
22:    $(\text{next}_i, \text{next}_j) \leftarrow \text{PROCHAINECASE}(i, j, n, m)$ 
23:
24:   if  $(\text{next}_i, \text{next}_j)$  invalide then
25:     if SOLUTIONTROUVÉE( $Na, NaM$ ) then
26:        $solutions.append(M)$ 
27:     end if
28:   else
29:      $E \leftarrow \text{ÉTATSSOISSIBLES}(M, NaM, Na, \text{next}_i, \text{next}_j)$ 
30:     for each  $v \in E$  do
31:       BACKTRACK( $\text{next}_i, \text{next}_j, v$ )
32:     end for
33:   end if
34:   RESTORENAM( $NaM, \text{restore}$ )
35:    $M[i][j] \leftarrow$  ancienne valeur
36: end function
```

La principale source de contradictions vient finalement de l'écart entre NaM construit par le backtracking au fur et à mesure et Na , le puzzle que l'on essaie de résoudre, en sachant le nombre de modifications qu'une case peut encore subir. Pour les détails d'implémentation de ÉTATSSOISSIBLES, nous invitons le lecteur à parcourir le fichier Slither-Link.py de la page [Github](#).

Performances Une des principales forces de cet algorithme se situe dans les puzzles qui contiennent beaucoup d’indices, car les contraintes sont alors très fortes. Sur une machine standard (processeur 1,1 GHz, RAM 1,1 Go, Python 3.12.3), l’algorithme résout une grille 900×900 avec toutes les cases pourvues d’un indice en environ 40 secondes, ce qui est très satisfaisant en comparaison à un solveur SAT classique (nous comparons les résultats dans la sous-section 4.3).

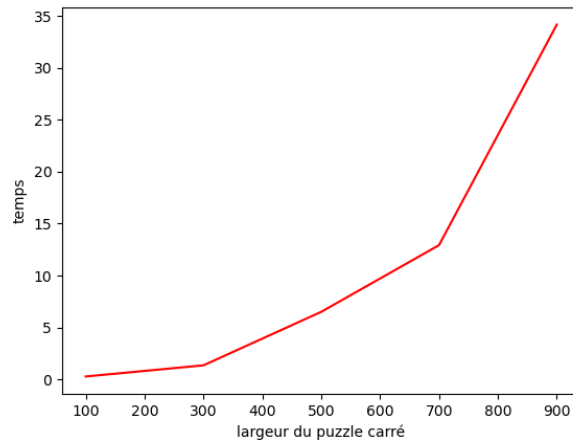


FIGURE 17 – Algorithme de backtracking sur des puzzles où toutes les cases sont remplies, de 100×100 à 900×900 (temps en secondes, échelle logarithmique)

Limite majeure Actuellement, la principale contrainte de l’algorithme se trouve dans la diminution du nombre d’indices et dans l’ordre de parcours imposé des cases. En effet, sur des cas extrêmes comme des 100×2 ou des 2×100 , les temps de résolution diffèrent énormément. Des développements d’heuristiques permettraient de fortement améliorer les capacités de ce solveur en orientant le puzzle de façon à optimiser le temps de calcul. Une première heuristique étudiée consiste à calculer la distance de la case $(0,0)$ au centre de gravité des cases non indicées. On remarque que plus la distance est faible, plus il y a d’incertitude sur le temps de résolution ; ceci reste une piste de recherche.

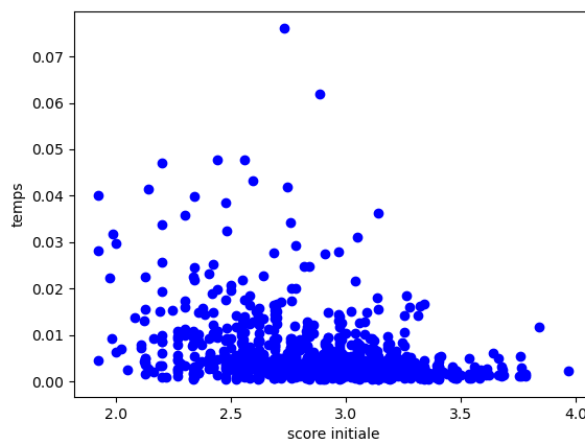


FIGURE 18 – Lien entre la distance du centre de gravité des cases non indicées et le temps de résolution sur un échantillon de 1000 puzzles 5×5 à 50% de cases masquées

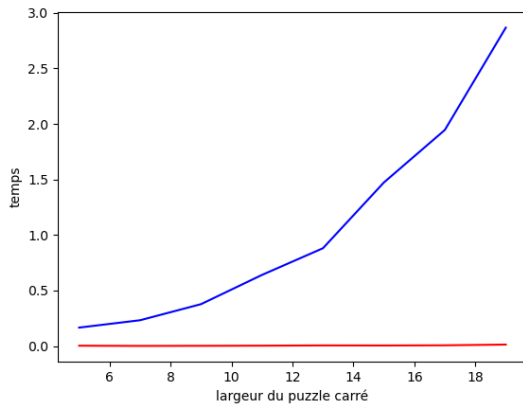
4.4 SMT solvers et comparaisons

Toutes les données ont été produites à partir de la même machine, dont voici les spécificités :

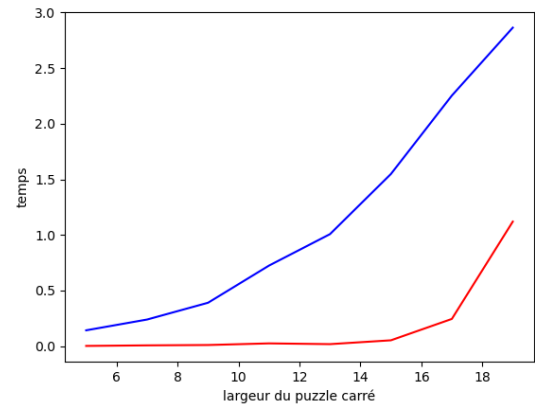
Processeur	1,1 GHz
RAM	1,1 Go
Version de Python	3.12.3
Version de Z3	3.16.0
Système d'exploitation	Ubuntu 24.04

TABLE 1 – Performances de la machine utilisée

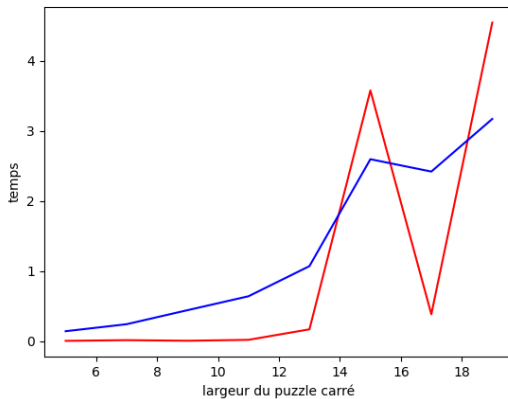
Nous avons modélisé le Slither Link en logique propositionnelle et utilisé le solveur Z3 pour des grilles jusqu'à 20×20 [4]. On remarque que le backtracking (rouge) original optimisé a des performances bien meilleures sur des puzzles contenant beaucoup d'indices. Cependant, le solveur Z3 (bleu) semble plus stable face à cette perte d'indices. Les pertes de performances de l'algorithme de backtracking peuvent s'expliquer par la force des contraintes lorsqu'il y a des indices. En revanche, les solveurs SMT n'arrivent pas à résoudre des puzzles de taille supérieure à 50×50 , contrairement à l'algorithme de backtracking qui peut franchir cette barrière lorsque le nombre d'indices est grand.



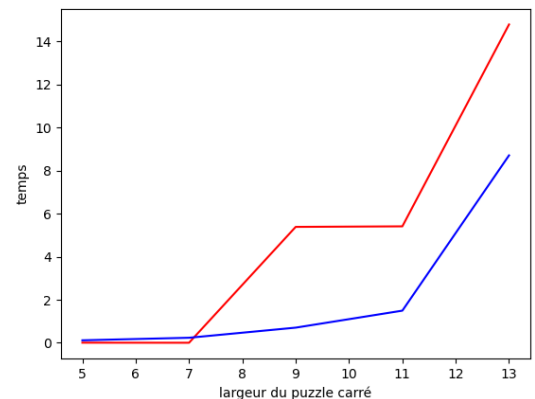
(a) 0% masqué



(b) 20% masqué



(c) 25% masqué



(d) 50% masqué

FIGURE 19 – Temps de résolution moyen en fonction de la taille du puzzle et du nombre d'indices masqués : backtracking (rouge) vs Z3 (bleu).

5 Généralisations et perspectives

Dans la Section 5.1, nous introduisons la notion de *CS-coloration* (Cycle Simple coloration) d'un graphe planaire G : elle généralise l'outil qui permet de générer des cycles simples dans le graphe dual intérieur G° via un critère local de modifiabilité. Ce critère est utile pour généraliser le processus de génération de puzzle Slither Link, point que nous développons dans la section 5.2.

5.1 CS-colorations et génération de cycles simples

Rappelons la construction duale dont nous aurons besoin.

Définition 2 (Graphe dual intérieur). *Soit $G = (V, E)$ un graphe planaire connexe muni d'un plongement planaire fixé π . On note $\mathcal{F}$ l'ensemble des faces de $\pi(G)$ et $f_\infty \in \mathcal{F}$ la face non bornée. Le graphe dual $G^* = (\mathcal{F}, E^*)$ a un sommet par face de $\pi(G)$ et une arête $e^* \in E^*$ pour chaque arête $e \in E$, reliant les deux faces qui bordent e . On appelle graphe dual intérieur de G le graphe $G^\circ = G^* \setminus \{f_\infty^*\}$.*

Définition 3 (Bordure et contour). *Soit $G = (V, E)$ un graphe planaire connexe avec plongement fixé π .*

- *La bordure de G , notée ∂G , est l'ensemble des sommets de V incidents à la face extérieure f_∞ .*
- *Pour $x \in V \setminus \partial G$, le contour de x , noté $\mathcal{C}_x$, est l'ensemble des sommets de $V \setminus \{x\}$ qui partagent une face bornée avec x dans $\pi(G)$.*

Définition 4 (CS-coloration). *Soit $G = (V, E)$ un graphe planaire connexe avec plongement fixé, tel que tout sommet de V possède au moins 3 voisins. Une CS-coloration de G est une fonction $\chi : V \rightarrow \{I, E\}$ telle que :*

- (CS1) *Tout sommet de la bordure ∂G est de couleur E .*
- (CS2) *Les sous-graphes induits $G_I = G[\chi^{-1}(I)]$ et $G_E = G[\chi^{-1}(E)]$ sont tous deux connexes.*

On dit que (G, χ) est un graphe CS-colorié.

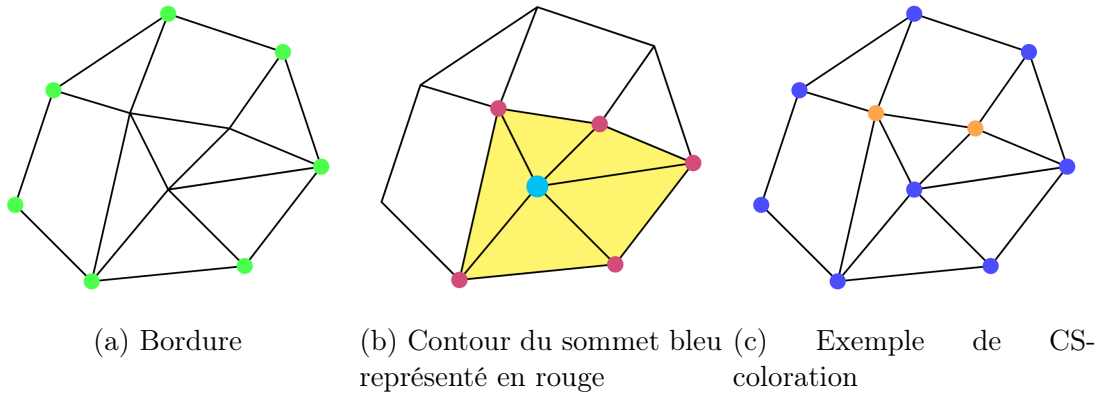


FIGURE 20 – Illustration des définitions sur un graphe particulier. L'extérieur est en bleu et l'intérieur en orange.

Définition 5 (Coupure et cycle dual d'une CS-coloration). Soit (G, χ) un graphe CS-colorié. On définit deux objets distincts.

- La coupure de χ , ensemble d'arêtes de G :

$$\partial_G(\chi) := \{uv \in E \mid \chi(u) \neq \chi(v)\} \subseteq E.$$

- Le cycle dual de χ , image de la coupure dans G^* :

$$\partial_{G^*}(\chi) := \{e^* \in E^* \mid e \in \partial_G(\chi)\} \subseteq E^*.$$

Ces deux ensembles encodent la même frontière géométrique entre G_I et G_E , mais dans deux graphes différents : $\partial_G(\chi)$ vit dans G (arêtes de la grille de points), tandis que $\partial_{G^*}(\chi)$ vit dans G° (la boucle que trace effectivement le joueur). La condition **(CS1)** garantit que $\partial_{G^*}(\chi)$ est bien inclus dans G° et non dans G^* tout entier.

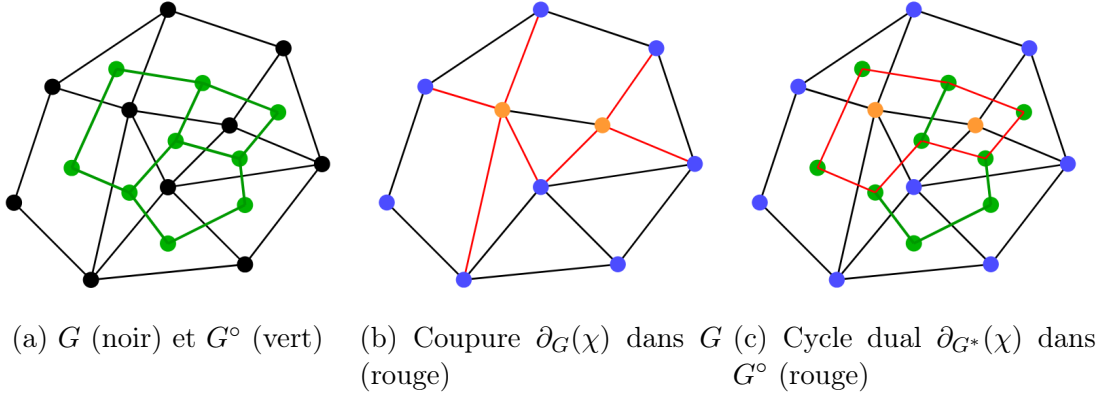


FIGURE 21 – La coupure vit dans G , le cycle dual vit dans G° : deux encodages du même objet géométrique dans deux graphes différents.

Remarques 1. Sur les conditions de CS-coloration

- **(CS1)** fixe canoniquement l'une des deux représentations : le cycle dual $\partial_{G^*}(\chi)$ est invariant par échange global de I et E (complémentation), et imposer que la bordure est toujours E lève cette ambiguïté. Géométriquement, l'extérieur du graphe est toujours à l'extérieur du cycle dans G° . Cette condition garantit également que le cycle dual vit dans G° et non dans G^* tout entier (voir Théorème 3).
- **(CS2)** garantit que le cycle dual $\partial_{G^*}(\chi)$ est un unique cycle simple dans G° et non une union de cycles disjoints.

Le théorème 3 (établi en section 6) montre que générer une CS-coloration de G revient exactement à générer un cycle simple dans G° .

Avec un critère de modifiabilité, nous montrons une façon astucieuse de générer un cycle simple dans un graphe planaire dont tous les sommets de la bordure sont au moins de degré 2 et les autres ont un degré au moins 3.

Définition 6 (Sommet modifiable). *Soit (G, χ) un graphe CS-colorié et $x \in V \setminus \partial G$. On dit que x est modifiable si :*

- (M1) *x possède au moins un voisin de couleur I et au moins un voisin de couleur E .*
- (M2) *Dans le contour $\mathcal{C}_x$, les sommets de couleur I forment une unique composante connexe, et de même pour les sommets de couleur E .*

On démontre dans la section 6 qu'un sommet x du graphe G est modifiable si et seulement si la CS-coloration χ à laquelle on a modifié la couleur de x reste une CS-coloration de G . Ceci justifie la correction des algorithmes de génération de solutions Slither Link (cycle simple) de la section 3 et généralise le principe de ces algorithmes à des graphes planaires dont tous les sommets ont degré au moins 3.

5.2 Application : Slither Link sur graphe planaire

Le Slither Link généralisé s'obtient en contraignant le cycle simple de G° à passer un nombre prescrit de fois autour de certains sommets de G° . Rappelons que les sommets de G° sont les faces bornées de G , et qu'une arête e^* de G° est incidente à un sommet f^* si et seulement si l'arête e de G correspondante borde la face f dans $\pi(G)$.

Définition 7 (Problème Slither Link généralisé). *Soit $G = (V, E)$ un graphe planaire connexe avec plongement fixé, dont tous les sommets ont degré au moins 3. Soit $\mathcal{F}^* = \mathcal{F} \setminus \{f_\infty\}$ l'ensemble des faces bornées de G , dont les sommets duaux f^* sont les sommets de G° . Soit $v : \mathcal{F}_v^* \rightarrow \{0, 1, 2, 3\}$ une fonction d'indices partielle définie sur un sous-ensemble $\mathcal{F}_v^* \subseteq \mathcal{F}^*$ des faces contraintes (les faces de $\mathcal{F}^* \setminus \mathcal{F}_v^*$ sont sans indice).*

Le problème SLITHER LINK sur G consiste à décider s'il existe une CS-coloration χ de G avec $G_I \neq \emptyset$ telle que, pour toute face contrainte $f \in \mathcal{F}_v^$, le cycle dual $\partial_{G^*}(\chi) \subseteq E(G^\circ)$ passe exactement $v(f)$ fois autour du sommet f^* de G° , c'est-à-dire :*

$$\deg_{\partial_{G^*}(\chi)}(f^*) = v(f).$$

Ici, $\deg_{\partial_{G^}(\chi)}(f^*)$ désigne le nombre d'arêtes de G° appartenant à $\partial_{G^*}(\chi)$ et incidentes au sommet f^* , autrement dit le nombre d'arêtes de G bordant la face f dont les deux extrémités sont de couleurs opposées dans χ .*

Remarques 2. *Tous les objets de cette définition vivent dans G° : les sommets contraints sont des sommets f^* de G° , le cycle dual $\partial_{G^*}(\chi)$ est un sous-ensemble d'arêtes de G° , et la contrainte $\deg_{\partial_{G^*}(\chi)}(f^*) = v(f)$ est une contrainte de degré dans G° . La CS-coloration χ , elle, vit sur G : c'est par la bijection du Théorème 3 qu'elle encode un cycle simple dans G° .*

Sur la grille rectangulaire, les sommets de G° sont les cases, les arêtes de G° incidentes à f^ sont les quatre côtés de la case f , et la contrainte $\deg_{\partial_{G^*}(\chi)}(f^*) = v(f)$ est exactement la règle du Slither Link classique : la boucle doit border exactement $v(f)$ côtés de la case f .*

La NP-complétude du problème généralisé découle simplement de la NP-complétude du sous-problème sur quadrillage carré et est établie en Section 6.

6 Preuves et résultats théoriques

Avant de commencer les démonstrations, nous introduisons un lemme nécessaire à la démonstration des théorèmes 3 et 2.

Théorème 1 (Propriété locale de connexité). *Soit $G = (V, E)$ un graphe planaire connexe muni d'une CS-coloration χ non triviale ($|G_I| \geq 2$). Alors pour tout sommet $x \in V \setminus \partial G$, le sous-graphe induit par $(\mathcal{C}_x \cup \{x\}) \cap \chi^{-1}(\chi(x))$ est connexe (reste vrai avec $|G_I| = 1$ mais n'a aucun intérêt).*

Démonstration. Soit $x \in V \setminus \partial G$ et supposons par l'absurde que le sous-graphe $H = G[(\mathcal{C}_x \cup \{x\}) \cap \chi^{-1}(\chi(x))]$ n'est pas connexe. Par symétrie, supposons $\chi(x) = E$. Donc $H \subseteq G_E$ est non connexe : il existe au moins deux composantes connexes H_1 et H_2 de H .

Comme $x \in H$ et H est non connexe, x appartient à l'une des composantes, disons H_1 . La composante H_2 contient donc au moins un sommet $y \in \mathcal{C}_x$ avec $\chi(y) = E$ et $y \neq x$, tel qu'il n'existe pas de chemin de x à y dans H .

Puisque $x \notin \partial G$, le sommet x n'est incident à aucune arête de la face extérieure f_∞ . Les faces incidentes à x dans $\pi(G)$ sont donc toutes des faces bornées, et elles forment un cycle de faces autour de x dans le plongement planaire. En parcourant ce cycle de faces dans l'ordre cyclique du plongement, on rencontre alternativement des portions de $\mathcal{C}_x \cap \chi^{-1}(I)$ et de $\mathcal{C}_x \cap \chi^{-1}(E)$.

Puisque H_1 et H_2 sont deux composantes connexes distinctes de $\mathcal{C}_x \cap \chi^{-1}(E) \cup \{x\}$, en parcourant le contour x , on passe nécessairement d'un sommet de $H_1 \cap \mathcal{C}_x$ à un sommet de $H_2 \cap \mathcal{C}_x$ en traversant au moins une portion de $\mathcal{C}_x \cap \chi^{-1}(I)$, puis on revient de H_2 à H_1 en traversant une autre portion de $\mathcal{C}_x \cap \chi^{-1}(I)$. Il y a donc au moins deux arcs de $\mathcal{C}_x \cap \chi^{-1}(I)$ qui séparent $H_1 \cap \mathcal{C}_x$ de $H_2 \cap \mathcal{C}_x$ dans le cycle de faces autour de x .

Considérons maintenant G_I . Chacun de ces arcs de $\mathcal{C}_x \cap \chi^{-1}(I)$ forme une portion connexe de G_I dans le voisinage de x . Par la planarité de G et la structure cyclique des faces autour de x , ces deux arcs sont séparés dans le plan par les portions de couleur E : l'un est « à gauche » de x et l'autre « à droite » dans le plongement $\pi(G)$.

Plus précisément, les arêtes de $\partial_G(\chi)$ incidentes à x forment un chemin fermé autour de x dans le plongement dual G^* . Si H_1 et H_2 sont deux composantes distinctes, ce chemin fermé contient au moins deux arcs de sommets de couleur I dans $\mathcal{C}_x$ séparés par des arcs de couleur E . Cela signifie que le cycle dual $\partial_{G^*}(\chi) \subseteq E(G^\circ)$ passe au moins deux fois autour du sommet dual x^* dans G° — mais $x \notin \partial G$, donc x ne correspond à aucun sommet de G° (les sommets de G° sont les faces bornées de G , pas les sommets de G).

Reformulons l'argument directement en termes de connexité. Les deux arcs de $\mathcal{C}_x \cap \chi^{-1}(I)$ appartiennent tous deux à G_I . Comme G est planaire et que ces deux arcs sont séparés dans le cycle de faces autour de x par des arcs de couleur E , tout chemin dans G_I reliant ces deux arcs doit soit passer par x (impossible car $\chi(x) = E$), soit contourner x en empruntant des arêtes n'appartenant pas à $\mathcal{C}_x$. Mais un tel chemin de contournement, combiné avec les deux arcs de $\mathcal{C}_x \cap \chi^{-1}(I)$ et les arêtes de $\partial_G(\chi)$ incidentes à x , formerait un cycle dans G qui sépare une partie de G_I du reste, contredisant la connexité de G_I imposée par (CS2).

On a donc une contradiction dans tous les cas : H est nécessairement connexe. $\square$

6.1 Démonstration du critère de modifiabilité généralisé

Théorème 2 (Modifiabilité sur graphe planaire). *Soit (G, χ) un graphe CS-colorié non trivial ($G_I \neq \emptyset$). On note $\bar{\chi}$ la coloration obtenue à partir de χ en échangeant la couleur de x . Alors :*

$$x \text{ est modifiable} \iff (G, \bar{\chi}) \text{ est un graphe CS-colorié avec } G_{\bar{\chi}, I} \neq \emptyset.$$

Démonstration. Soit G un graphe muni de χ une CS-coloration.

Sens direct ($\Rightarrow$). Supposons x modifiable. La condition **(CS1)** est préservée car $x \notin \partial G$ et le degré des sommets ne change pas. Par symétrie, supposons $\chi(x) = E$.

Connexité de $G_{\bar{\chi}, I} = G_I \cup \{x\}$. Par **(M1)**, x possède un voisin y de couleur I . L'arête xy relie x à G_I , donc $G_I \cup \{x\}$ est connexe.

Connexité de $G_{\bar{\chi}, E} = G_E \setminus \{x\}$. Soient $s, t \in G_E \setminus \{x\}$ et $s = v_0, \dots, v_k = t$ un chemin dans G_E . Si le chemin évite x , rien à faire. S'il passe par x , soient v_{i-1} et v_{i+1} les voisins de x sur ce chemin : ils appartiennent à $\mathcal{C}_x \cap \chi^{-1}(E)$. Par **(M2)**, les sommets de couleur E dans $\mathcal{C}_x$ forment une unique composante connexe, donc il existe un chemin de v_{i-1} à v_{i+1} dans $\mathcal{C}_x \cap \chi^{-1}(E) \subseteq G_E \setminus \{x\}$. En substituant ce chemin au passage par x , on obtient un chemin de s à t dans $G_E \setminus \{x\}$. La condition **(CS2)** est alors vérifiée.

Sens réciproque ($\Leftarrow$). Procédons par contraposée. Supposons x non modifiable.

Cas 1 : (M1) non satisfaite. Par symétrie, supposons que tous les voisins de x sont de couleur E et $\chi(x) = E$. Alors $\bar{\chi}(x) = I$ et $G_{\bar{\chi}, I} = \{x\}$ est un singleton. Par le lemme 1, x a au moins 3 voisins, tous de couleur E dans $\bar{\chi}$, donc $\partial(\bar{\chi})$ contient au moins 3 arêtes incidentes à x . Or un cycle simple impose le degré 2, donc $\partial(\bar{\chi})$ n'est pas un cycle simple. Par le Théorème 3, $(G, \bar{\chi})$ n'est pas une CS-coloration valide.

Cas 2 : (M2) non satisfaite. Supposons que les sommets de couleur E dans $\mathcal{C}_x$ forment au moins deux composantes connexes $\mathcal{C}_{E_1}$ et $\mathcal{C}_{E_2}$, et que $\chi(x) = E$. Il faut montrer que $G_{\bar{\chi}, E} = G_E \setminus \{x\}$ n'est pas connexe.

Soient $c_1 \in \mathcal{C}_{E_1}$ et $c_2 \in \mathcal{C}_{E_2}$. Par lemme 1 appliqué à x dans χ , le sous-graphe $(\mathcal{C}_x \cup \{x\}) \cap \chi^{-1}(E)$ est connexe, donc c_1 et c_2 sont tous deux reliés à x dans ce sous-graphe : x est le seul pont entre $\mathcal{C}_{E_1}$ et $\mathcal{C}_{E_2}$ au niveau local.

Supposons par l'absurde qu'il existe un chemin $c_1 = v_0, \dots, v_k = c_2$ dans $G_E \setminus \{x\}$. Ce chemin évite x mais relie deux composantes connexes distinctes de $\mathcal{C}_x \cap \chi^{-1}(E)$, donc il doit sortir de $\mathcal{C}_x$. Or, tout chemin dans $G_E \setminus \{x\}$ sortant de $\mathcal{C}_x$ et y revenant emprunte nécessairement des arêtes passant par des faces non incidentes à x . Un tel chemin, combiné avec les connexions de c_1 et c_2 à x via $\mathcal{C}_x$, formerait un cycle dans G séparant une composante de $G_I \cap \mathcal{C}_x$ du reste de G_I , contredisant la connexité de G_I imposée par **(CS2)**.

Donc $G_{\bar{\chi}, E}$ n'est pas connexe et $(G, \bar{\chi})$ n'est pas une CS-coloration. $\square$

6.2 Correspondance cycle / CS-coloration

Théorème 3 (Correspondance cycle dans G° / CS-coloration de G). *Soit $G = (V, E)$ un graphe planaire connexe avec plongement fixé, dont tous les sommets ont degré au moins 3. Il existe une bijection entre :*

1. les cycles simples $C^* \subseteq E^\circ$ de G° tels que f_∞^* n'est incident à aucune arête de C^* ;
2. les CS-colorations χ de G avec $G_I \neq \emptyset$,

Démonstration. Nous montrons les deux sens de la bijection, en précisant à chaque étape dans quel graphe les objets vivent.

(CS-coloration de $G \Rightarrow$ cycle simple dans G° .) Soit χ une CS-coloration de G avec $G_I \neq \emptyset$. Montrons que $C^* = \partial_{G^*}(\chi) \subseteq E^*$ est un cycle simple dans G° .

Déjà $C^* \neq \emptyset$ car $G_I \neq \emptyset$ et $G_E \neq \emptyset$, et G est connexe, donc il existe $e \in \partial_G(\chi)$. Ensuite C^* est inclus dans G° . Supposons par l'absurde qu'il existe une arête $e^* \in \partial_{G^*}(\chi)$ qui admet comme extrémité f_∞^* ; alors cela signifie qu'il existe une arête dans $\partial_G(\chi)$ qui lie deux sommets de G de la bordure. Or par **(CS1)** cela est impossible car tous les nœuds de la bordure sont de la même couleur. Montrons à présent que chaque sommet de C^* admet exactement deux voisins dans C^* .

Soit f^* un sommet de G° . On considère l'ensemble que l'on note $\mathcal{E}$ des sommets de G qui entourent f^* . Montrons que les composantes I et E sont connexes dans $\mathcal{E}$. Supposons qu'il existe x, y deux sommets de $\mathcal{E}$ de même couleur. 1er cas : s'ils sont voisins, c'est ok. 2ème cas : si x et y ne sont pas voisins, par construction $y \in \mathcal{C}_x$; ensuite les sommets de $\mathcal{C}_x \setminus \mathcal{E}$ ne sont pas connectés à y . Ainsi d'après le lemme 1 il existe un chemin dans $\mathcal{C}_x \cap \chi^{-1}(E)$ qui lie x et y . Finalement $\mathcal{C}_x \cap \chi^{-1}(E)$ et $\mathcal{E} \cap \chi^{-1}(E)$ forment deux composantes connexes donc il y a exactement 2 arêtes liant deux sommets de $\mathcal{E}$ de couleur distincte, donc le sommet f^* est de degré 2 dans C^* .

La connexité de C^* découle de la propriété **(CS2)** avec le théorème de Jordan (s'il existait deux cycles, cela briserait la connexité de G_E ou G_I).

(Cycle simple dans $G^\circ \Rightarrow$ CS-coloration de G .) Soit $C^* \subseteq E^\circ$ un cycle simple dans G° tel que f_∞^* n'est incident à aucune arête de C^* . Les arêtes de G correspondantes forment $\partial_G = \{e \in E \mid e^* \in C^*\}$. Par l'analogue discret du théorème de Jordan appliqué à C^* dans G° , ce cycle sépare les sommets de G° (les faces bornées de G) en deux ensembles connexes. On étend cette séparation aux sommets de G : un sommet $x \in V$ est dit *intérieur* s'il est entouré par C^* (c'est-à-dire si toutes les faces incidentes à x sont du côté intérieur de C^*), et *extérieur* sinon. On pose $\chi(x) = I$ ou E en conséquence. La condition **(CS1)** est satisfaite car, par hypothèse, f_∞^* est à l'extérieur de C^* , donc tous les sommets de ∂_G (incidentes à f_∞^*) sont extérieurs. La connexité de G_I et G_E découle de la séparation de Jordan dans G° . Enfin, $\partial_{G^*}(\chi) = C^*$ par construction.

(Bijektivité.) Les deux constructions sont inverses l'une de l'autre, et l'ambiguïté de complémentation est levée par **(CS1)**.

(Conservation des indices.) Le degré de f^* dans C^* est le nombre d'arêtes de C^* incidentes à f^* dans G° , c'est-à-dire le nombre d'arêtes de G bordant f qui appartiennent à $\partial_G(\chi)$, c'est-à-dire le nombre de côtés de la face f séparant une couleur intérieure d'une couleur extérieure. C'est exactement la contrainte d'indice $v(f)$ dans la formulation classique. $\square$

6.3 NP-complétude du Slither Link

Takayuki Yato a démontré en 2000 que le Slither Link est NP-complet [1]. Nous présentons ici cette preuve en détail. Elle repose sur une réduction polynomiale depuis le problème du *cycle hamiltonien* dans les graphes planaires cubiques, lui-même NP-complet [9].

Théorème 4 (Yato [1]). *Le problème SLITHER LINK est NP-complet.*

L'appartenance à NP est immédiate : étant donné un ensemble d'arêtes $C \subseteq E$, on vérifie en temps polynomial que C est un cycle simple et que les contraintes d'indices sont satisfaites. La NP-difficulté est établie par la réduction décrite ci-dessous.

Le problème source : cycle hamiltonien dans les graphes planaires cubiques

On rappelle qu'un graphe est *cubique* si tous ses sommets sont de degré exactement 3, et *planaire* s'il peut être dessiné dans le plan sans croisement d'arêtes. Le problème HAMILTONIEN-CUBIQUE-PLANAIRE demande : étant donné un tel graphe $H = (V_H, E_H)$, existe-t-il un cycle passant exactement une fois par chaque sommet ? Ce problème est NP-complet [9].

Principe de la réduction

On construit, à partir de tout graphe planaire cubique G , une instance de Slither Link (une grille avec des indices) telle que les solutions de ce puzzle correspondent bijectivement aux cycles hamiltoniens de G . La construction est polynomiale en $|V_H| + |E_H|$ et s'effectue en trois étapes.

Étape 1 : projection de G sur une grille. Puisque G est planaire, on peut plonger ses n sommets et arêtes sur une grille entière $(n-2) \times (n-2)$ en $\mathcal{O}(n)$ [13], en ne conservant que des segments horizontaux et verticaux, de sorte que les arêtes ne se croisent qu'aux sommets. On obtient ainsi une représentation rectiligne de G sur la grille.

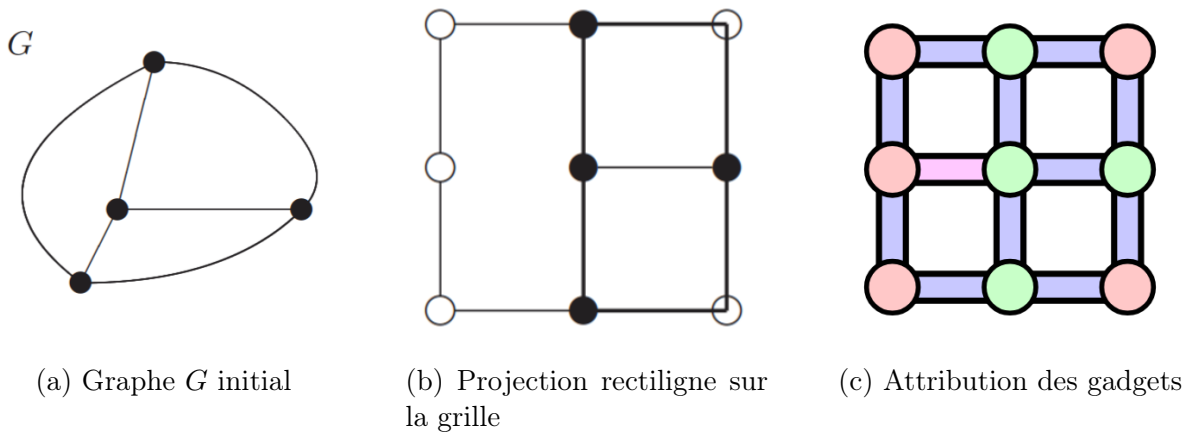


FIGURE 22 – Les trois étapes de la transformation de G en instance de Slither Link.

Étape 2 : les gadgets. On remplace chaque sommet et chaque arête de G par un *gadget* — un motif local de cases avec indices — qui encode la contrainte correspondante. Quatre types de gadgets sont utilisés.

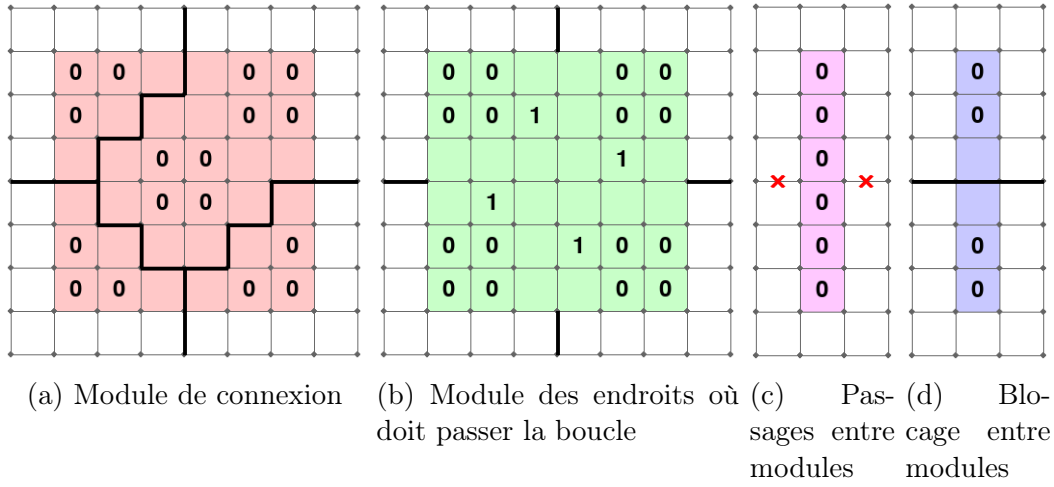


FIGURE 23 – Les quatre gadgets de la réduction.

Étape 3 : assemblage et instance finale. On assemble les gadgets sur la grille selon la projection rectiligne de G . On obtient une grille de Slither Link dont les indices sont entièrement déterminés par la structure de G .

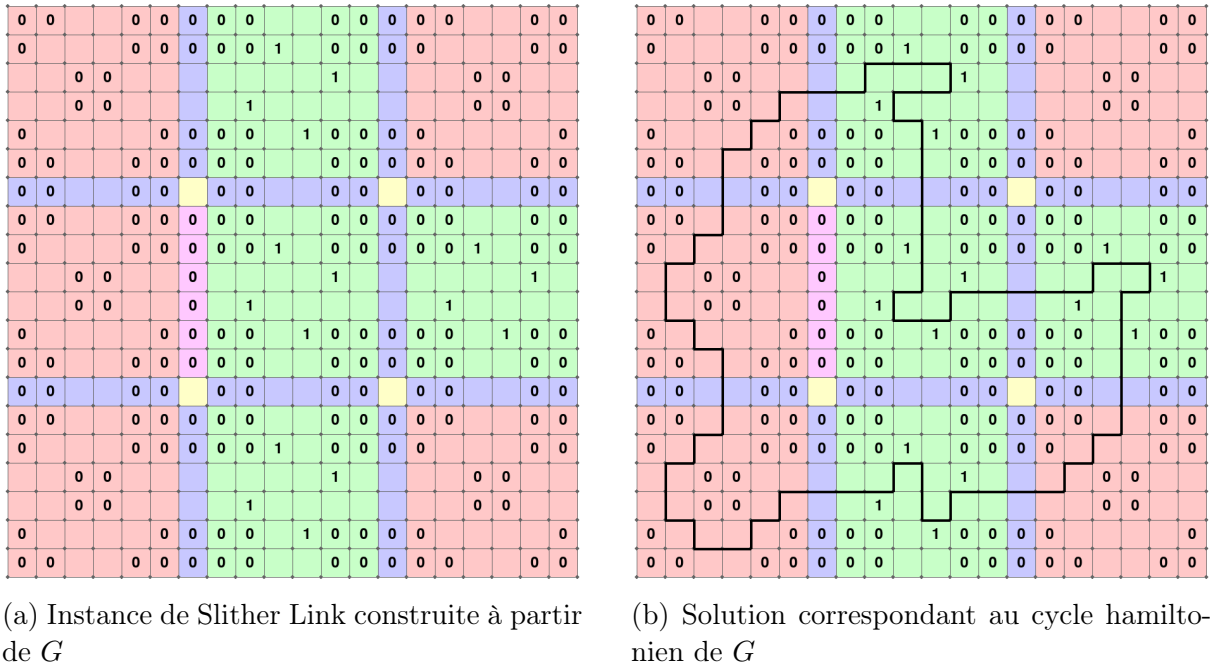


FIGURE 24 – Exemple complet de la réduction : à gauche, la grille de Slither Link construite ; à droite, la boucle solution, qui correspond bijectivement au cycle hamiltonien de G .

Correction de la réduction

Il reste à vérifier que la réduction est correcte, c'est-à-dire que G admet un cycle hamiltonien si et seulement si l'instance de Slither Link construite admet une solution.

($\Rightarrow$) Supposons que G admet un cycle hamiltonien $\mathcal{H}$. On construit une boucle C dans la grille de Slither Link comme suit : pour chaque arête de E_H empruntée par $\mathcal{H}$, on fait passer C par le module de passage obligatoire correspondant ; pour chaque sommet de V_H , on fait passer C par les deux couloirs du module de connexion correspondant aux deux arêtes de $\mathcal{H}$ incidentes à ce sommet. Comme $\mathcal{H}$ est un cycle simple passant par tous les sommets de G , la boucle C ainsi construite est un cycle simple dans la grille. Les contraintes d'indices de chaque gadget sont satisfaites par construction. Donc C est une solution valide.

($\Leftarrow$) Supposons que la grille de Slither Link admet une boucle solution C . La structure des gadgets force C à se comporter localement de la façon prescrite : au niveau de chaque module de connexion, C entre exactement par deux des trois couloirs ; au niveau de chaque module de passage obligatoire ou de blocage, C passe ou non de façon déterminée. L'ensemble des arêtes de G correspondant aux couloirs empruntés par C forme donc un sous-graphe de G où chaque sommet est de degré 2 (exactement deux couloirs par module de connexion) et connexe (car C est un cycle simple connexe). Un sous-graphe 2-régulier connexe d'un graphe est un cycle hamiltonien si et seulement s'il passe par tous les sommets : c'est bien le cas ici car chaque module de connexion est traversé. Donc G admet un cycle hamiltonien.

La taille de l'instance construite est polynomiale en $|V_H| + |E_H|$ car chaque gadget est de taille constante et le nombre de gadgets est proportionnel au nombre de sommets et d'arêtes de G . La réduction est donc bien polynomiale, ce qui conclut la preuve de NP-difficulté et donc la preuve du Théorème 4. $\square$

Question ouverte. Une question soulevée par nos expériences (Section 4) est la suivante : la NP-complétude est-elle préservée lorsque *toutes* les cases contiennent un indice, c'est-à-dire lorsque $\mathcal{F}_v^* = \mathcal{F}^*$? Nos mesures suggèrent que le jeu devient significativement plus facile en pratique dans ce régime, mais la réponse théorique reste ouverte. Une réponse positive nécessiterait de construire une réduction dans laquelle les gadgets de blocage (Figure 23d) peuvent être remplacés par des gadgets où toutes les cases sont contraintes, ce qui semble difficile compte tenu de la rigidité imposée.

7 Conclusion

Nous avons présenté une étude complète du Slither Link, de la génération de puzzles à leur résolution algorithmique, en passant par des résultats théoriques fondateurs. La modélisation par 2-coloration et le critère de modifiabilité (démontré rigoureusement pour tout graphe planaire) offrent une base solide pour produire des grilles variées et pour étendre le jeu à des supports non rectangulaires. L’algorithme de backtracking original, récemment optimisé, surpasse les solveurs SMT pour les grandes tailles, à condition que toutes les cases soient indicées ; des heuristiques d’ordre de parcours sont proposées pour améliorer ses performances sur les puzzles creux.

Sur le plan théorique, nous avons formalisé la notion de CS-coloration, établi son équivalence avec les cycles simples dans le graphe dual, et fourni une preuve complète du théorème de modifiabilité. La NP-complétude du Slither Link est détaillée à partir de la réduction de Yato. Des perspectives prometteuses incluent la généralisation aux graphes planaires quelconques, l’étude de l’unicité de la solution (ASP-complétude), et l’extension à des puzzles multidimensionnels ou multicolores.

```

1 >>> backtrack_solver([[2,2,2],[2,3,3],[3,1,1]])
2 -- etape 1 : indice = 0 0
3 [True , False, False] [4, 1, 0] [2, 2, 2]
4 [False, False, False] [1, 0, 0] [2, 3, 3]
5 [False, False, False] [0, 0, 0] [3, 1, 1]
6 -- etape 2 : indice = 0 1
7 [True , True , False] [3, 3, 1] [2, 2, 2]
8 [False, False, False] [1, 1, 0] [2, 3, 3]
9 [False, False, False] [0, 0, 0] [3, 1, 1]
10 -- etape 3 : indice = 1 0
11 [True , True , False] [2, 3, 1] [2, 2, 2]
12 [True , False, False] [3, 2, 0] [2, 3, 3]
13 [False, False, False] [1, 0, 0] [3, 1, 1]
14 -- etape 4 : indice = 0 2
15 [True , True , True ] [2, 2, 3] [2, 2, 2]
16 [True , False, False] [3, 2, 1] [2, 3, 3]
17 [False, False, False] [1, 0, 0] [3, 1, 1]
18 -- etape 5 : indice = 1 1
19 [True , True , True ] [2, 2, 3] [2, 2, 2]
20 [True , False, False] [3, 2, 1] [2, 3, 3]
21 [False, False, False] [1, 0, 0] [3, 1, 1]
22 -- etape 6 : indice = 2 0
23 [True , True , True ] [2, 2, 3] [2, 2, 2]
24 [True , False, False] [2, 2, 1] [2, 3, 3]
25 [True , False, False] [3, 1, 0] [3, 1, 1]
26 -- etape 7 : indice 1 2
27 [True , True , True ] [2, 2, 2] [2, 2, 2]
28 [True , False, True ] [2, 3, 3] [2, 3, 3]
29 [True , False, False] [3, 1, 1] [3, 1, 1]
30 --
31 Solution trouvee :
32 [True , True , True ]
33 [True , False, True ]
34 [True , False, False]

```

Références

- [1] T. Yato, *Complexity and completeness of finding another solution and its application to puzzles*, 2003. <https://zanotti.univ-tln.fr/ALGO/I51/SudokuNP-complet.pdf>
- [2] J. Olson, *How Slitherlink Should be Solved*. <https://jonathanolson.net/slitherlink/>
- [3] D. Anderson, D. Hamalainen, E. Kwiatkowski, V. Lambert, *Slithering the Link*. https://cs.carleton.edu/cs_comp/s1516/slither/final-results/presentation.pdf
- [4] D. Westreicher, *Slitherlink Reloaded*, 2011. <https://david-westreicher.github.io/static/papers/ba-thesis.pdf>
- [5] P. Lafourcade, D. Miyahara, T. Mizuki, T. Sasaki, H. Sone, *A Physical ZKP for Slitherlink : How to Perform Physical Topology-Preserving Computation*, 2024. <https://uca.hal.science/hal-04402827v1/document>
- [6] R. Fournier-S’Niehota, *Cnam, visualisation de graphes*, 2020. <https://cedric.cnam.fr/vertigo/Cours/RCP216/docs/RCP216-ReseauxSociaux3.pdf>
- [7] N. Beluhov, *Slitherlink Signatures*, <https://arxiv.org/pdf/2308.08798>.
- [8] R. Yoshinaka, T. Saitoh, J. Kawahara, K. Tsuruma, H. Iwashita, S. Minato, *Finding All Solutions and Instances of Numberlink and Slitherlink by ZDDs*, MDPI, 5(2), 176.
- [9] R.M. Karp, *On the Computational Complexity of Combinatorial Problems*, 1975.
- [10] S.A. Cook, *The complexity of theorem-proving procedures*, in Proceedings of the 3rd ACM STOC, 1971, pp. 151–158.
- [11] J. Kölker, *Selected Slither Link Variants are NP-complete*, 2012. https://www.jstage.jst.go.jp/article/ipsjjip/20/3/20_709/_pdf
- [12] C. Gong, *Slitherlink on Triangular Grids*, 2026. <https://arxiv.org/pdf/2410.19078>
- [13] W. Schnyder, *Embedding planar graphs on the grid*, 1990. <https://dl.acm.org/doi/epdf/10.5555/320176.320191>

Github

<https://github.com/nohl4n/TIPE-slither-link>